Linux PAM: A Arquitetura da Identidade

Guia Teórico e Prático | Segurança e Autenticação Avançada no Proxmox

1. Introdução: O Desafio da Autenticação

Num sistema operativo moderno, existem dezenas de formas de entrar: consola local, SSH, FTP, interfaces web. Historicamente, se quiséssemos mudar o método de autenticação (por exemplo, passar de passwords simples para reconhecimento biométrico ou bases de dados LDAP), tínhamos de reescrever e recompilar o código de cada uma destas aplicações. O **Pluggable Authentication Modules (PAM)** resolve este problema.

O PAM atua como uma camada intermédia dinâmica (um *middleware*) entre as aplicações e os mecanismos de autenticação. Com o PAM, o serviço SSH já não verifica senhas. Ele simplesmente "pergunta" ao PAM: *"O utilizador 'admin' tem permissão para aceder?"*. O PAM executa a sua lista de regras e devolve o veredicto.

O Foco deste Workshop: O PAM é vasto, podendo controlar desde limites de uso de CPU até montagem de discos. Neste workshop, vamos focar-nos em três pilares essenciais da defesa de um servidor em produção:

- **MFA (Multi-Factor Authentication):** Como adicionar um Google Authenticator ao SSH.
- **Defesa contra Força Bruta:** Como bloquear contas automaticamente sob ataque.
- **Qualidade de Passwords:** Como impedir que utilizadores escolham senhas fracas.

2. Anatomia de uma Decisão PAM

O coração do PAM reside em como ele encadeia múltiplos testes (módulos). Para que uma autenticação seja bem-sucedida, o pedido tem de atravessar vários "grupos de gestão", e cada módulo dentro desse grupo emite uma recomendação baseada numa "flag de controlo".

3. Os Quatro Grupos de Gestão (Facilities)

Quando o SSH pede autorização, o PAM processa o pedido em quatro fases distintas, pela seguinte ordem:

Grupo / Facility	Função no Processo de Login
auth	**Verificação de Identidade:** É aqui que se pedem passwords, chaves biométricas ou tokens gerados por apps (MFA). Estabelece *quem* a pessoa é.
account	**Validação de Acesso:** Mesmo com a senha certa, o utilizador pode entrar? Este grupo verifica se a conta expirou, se está bloqueada por ataques, ou se o acesso está restrito a certas horas do dia.
session	**Preparação do Ambiente:** Executado antes e depois do utilizador entrar. Pode montar diretórios, mostrar a "Mensagem do Dia" (MOTD) ou registar o login no ficheiro de logs do sistema.
password	**Atualização de Tokens:** Invocado quando uma password expira e o utilizador é forçado a criar uma nova antes de ter acesso ao terminal.

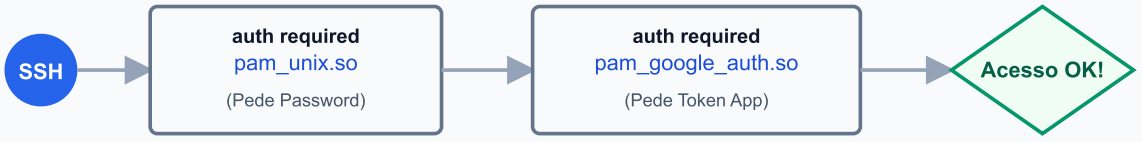
4. Flags de Controlo (Control Flags)

Dentro de cada grupo, configuramos módulos. A *Flag de Controlo* diz ao PAM a importância de cada módulo. Isto é o que permite combinar senhas e MFA na perfeição:

Flag	Comportamento (Lógica da Máquina de Estados)
required	O sucesso é obrigatório. Contudo, **se falhar, o PAM não rejeita de imediato**. Ele continua a correr os restantes módulos do grupo em segredo e só no fim rejeita. Isto previne que atacantes descubram exatamente em que passo do login falharam.
requisite	O sucesso é estritamente obrigatório. Se este módulo falhar, o PAM **rejeita o login imediatamente** sem correr mais testes. Usa-se para poupar recursos ou evitar pedidos de senha desnecessários.
sufficient	Se o módulo passar (e nenhum `required` anterior tiver falhado), o PAM **aprova a fase imediatamente**, saltando módulos seguintes.
optional	Usado para módulos que fazem tarefas passivas, como registar logs. O seu sucesso/falha não dita o resultado da autenticação.

5. Arquitetura de Empilhamento: O Caso do MFA

Como é que o PAM obriga o utilizador a fornecer **tanto** a password **como** o código do telemóvel? Ele faz isso "empilhando" módulos no grupo `auth`, ambos com a flag `required`. Vejamos o fluxo quando o utilizador tenta aceder por SSH:



Se o utilizador acertar na password (`pam_unix.so`), o fluxo passa para o próximo módulo. Se inserir o token corretamente (`pam_google_auth.so`), o fluxo termina com sucesso e o utilizador entra. O facto de ambos serem `required` garante que a quebra de apenas um dos fatores não compromete o sistema.

O Mito do MFA: É preciso um email da Google?
Um erro comum entre iniciantes é pensar que este módulo exige ligação à internet ou um email da Google associado ao Linux. **Não é necessário nenhum email!** O sistema funciona através do standard aberto **TOTP (Time-based One-Time Password)**, de forma 100% offline. O Linux desenha um enorme *QR Code usando caracteres de texto (ASCII)* diretamente no terminal. O utilizador aponta a câmara do telemóvel (usando a app Google Authenticator, Authy ou Microsoft) e a validação passa a ser feita localmente através de matemática sincronizada com o relógio do servidor, garantindo total privacidade e independência da nuvem.

Se a password estivesse errada, o PAM (devido à flag `required`) *continuaria* a pedir o código do Google Authenticator na mesma, rejeitando o acesso apenas no final. Isto impede que um atacante saiba se descobriu a password correta ou não.

6. Parte Prática 1: Reforço da Linha de Base

🚨 **CRÍTICO ANTES DE COMEÇAR:** Vamos editar regras de autenticação no servidor Linux (Proxmox). Um erro nestes ficheiros bloqueia o servidor. 1. No Proxmox, faça um **Snapshot** ("Base_Limpa") da VM. 2. Mantenha uma consola noVNC aberta logada como root como via de emergência.

Exercício A: Obrigar Passwords Fortes (pam_pwquality)

Vamos garantir que nenhuma password nova no sistema é fraca. O módulo verifica se a password não é baseada em dicionários e se cumpre métricas de complexidade.

1. Instale o módulo: `sudo apt install libpam-pwquality`
2. Edite o ficheiro central de passwords: `sudo nano /etc/pam.d/common-password`
3. Modifique a linha do módulo para exigir 12 caracteres (minlen) e rejeitar falhas de imediato (requisite):

```
password requisite pam_pwquality.so retry=3 minlen=12 ucredit=-1
```

Exercício B: Bloqueio de Ataques Brute-Force (pam_faillock)

Os ataques mais comuns ao SSH são bots a tentar palavras-passe repetidamente. Vamos bloquear qualquer conta que falhe a senha 3 vezes seguidas.

1. Edite as regras de autenticação: `sudo nano /etc/pam.d/common-auth`
2. Adicione as regras de bloqueio no topo do ficheiro:

```
auth required pam_faillock.so preauth silent deny=3 unlock_time=120 ... (as restantes linhas ficam aqui) ... auth [default=die] pam_faillock.so authfail deny=3 unlock_time=120
```

3. Edite as regras de conta: `sudo nano /etc/pam.d/common-account`
4. Adicione no final: `account required pam_faillock.so`

7. Parte Prática 2: Implementar MFA no SSH

Este é o cenário avançado do nosso workshop. Vamos forçar o serviço SSH a exigir um token gerado na app (Google Authenticator / Authy) além da palavra-passe padrão do Linux.

1. **Instalar a ferramenta no servidor:**
   ```
   sudo apt update sudo apt install libpam-google-authenticator
   ```
2. **Gerar o código (Feito com o utilizador normal):**
   Corra o comando abaixo no terminal com a conta do utilizador (não como root). O terminal irá gerar um enorme QR Code em ASCII art. Abra o Google Authenticator no seu telemóvel e leia o ecrã.
   ```
   google-authenticator
   ```
   Responda 'y' a todas as perguntas de segurança que o assistente fizer.
3. **Configurar o PAM para o SSH:**
   Agora temos de dizer ao PAM para empilhar este módulo especificamente no acesso SSH.
   ```
   sudo nano /etc/pam.d/sshd
   ```
   Procure a linha `@include common-auth` (que pede a password). Logo por baixo dela, adicione:
   ```
   auth required pam_google_authenticator.so
   ```
4. **Avisar o SSH para aceitar tokens:**
   Por fim, o serviço SSH (A Porta) precisa de saber que vai ter de apresentar novos *prompts* interativos ao utilizador.
   ```
   sudo nano /etc/ssh/sshd_config
   ```
   Encontre (ou adicione) a linha e mude para `yes`:
   ```
   KbdInteractiveAuthentication yes
   ```
5. Reinicie o serviço: `sudo systemctl restart ssh`

O Teste Final: Abra uma linha de comandos no seu computador cliente e faça `ssh utilizador@ip_servidor`. O sistema irá pedir a sua "Password" e, logo a seguir, o "Verification code". O servidor agora está protegido por MFA.